Agenda.

- → Copy Constructor.
- → Shallow Copy (vs) Deep Copy
- → Pass By Value (vs) Pass By Reference
- → Inheritance
- → Polymorphism.

Constructor

⇒ Special function in the class which is responsible for object creation.

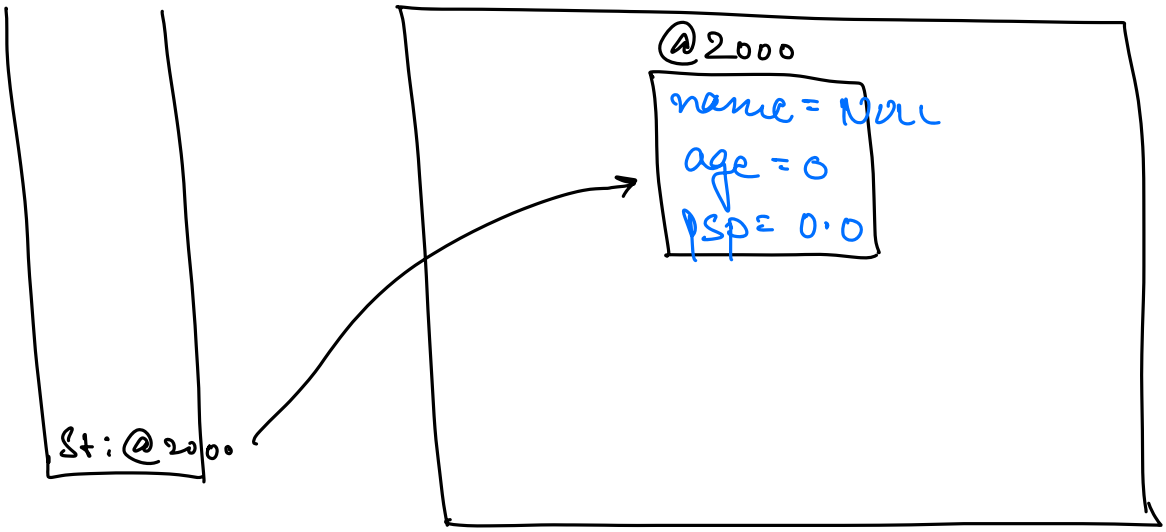
⇒ Same name as of the class name.

⇒ No return type.

1) Default Constructor

```
Student {
    name
    age
    Psp
}
```

```
Student st = new Student ()
```



2) Custom | Parameterised Constructor

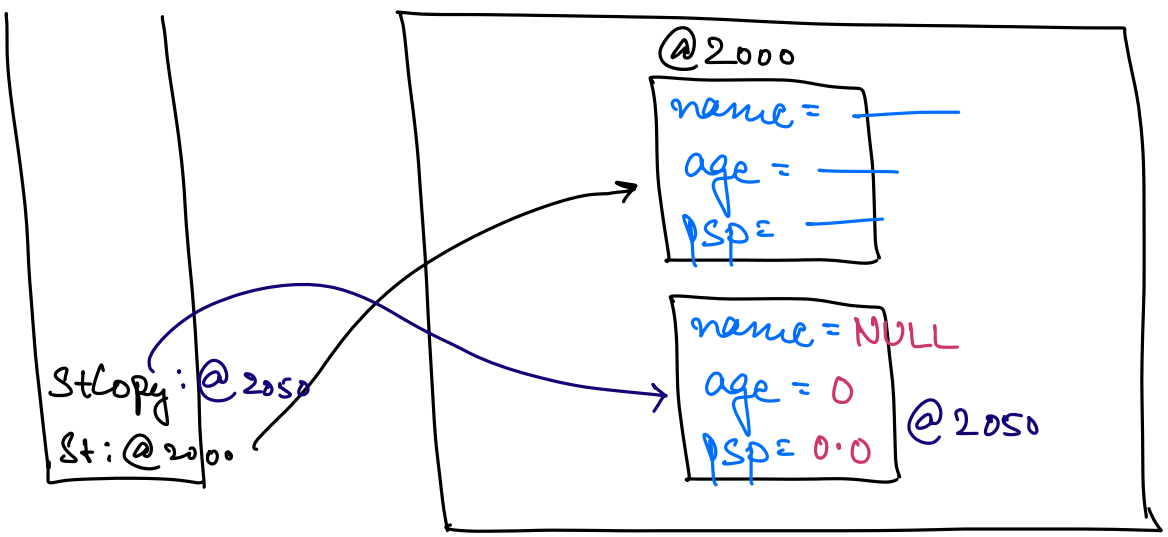
```
Student {
    name
    age
    psp
    Student (name, age, psp) {
        this.name = name;
        this.age = age;
        this.psp = psp;
    }
}
```

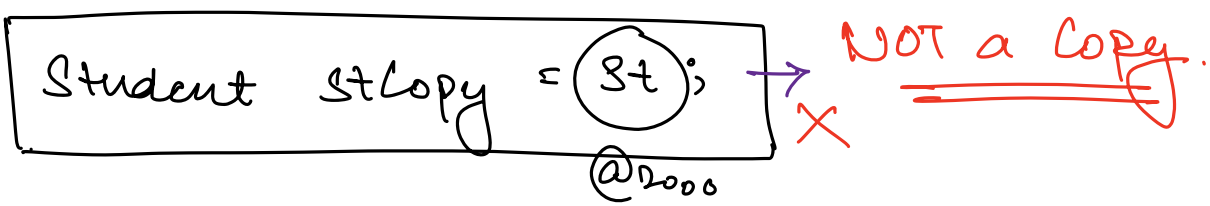
An object with the default value gets created Before even the first line of the Custom Constructor gets executed & then the values | attrs gets initialized.

3) Given an object of the class, create the copy of this object.

→ A new object with exact same set of attrs.

```
Student st = new Student()
st.name = "Vishal"
st.age = 24
st.psp = 94.0
```






```
Student stCopy = new Student();
stCopy.name = st.name;
stCopy.age = st.age;
stCopy.psp = st.psp;
```

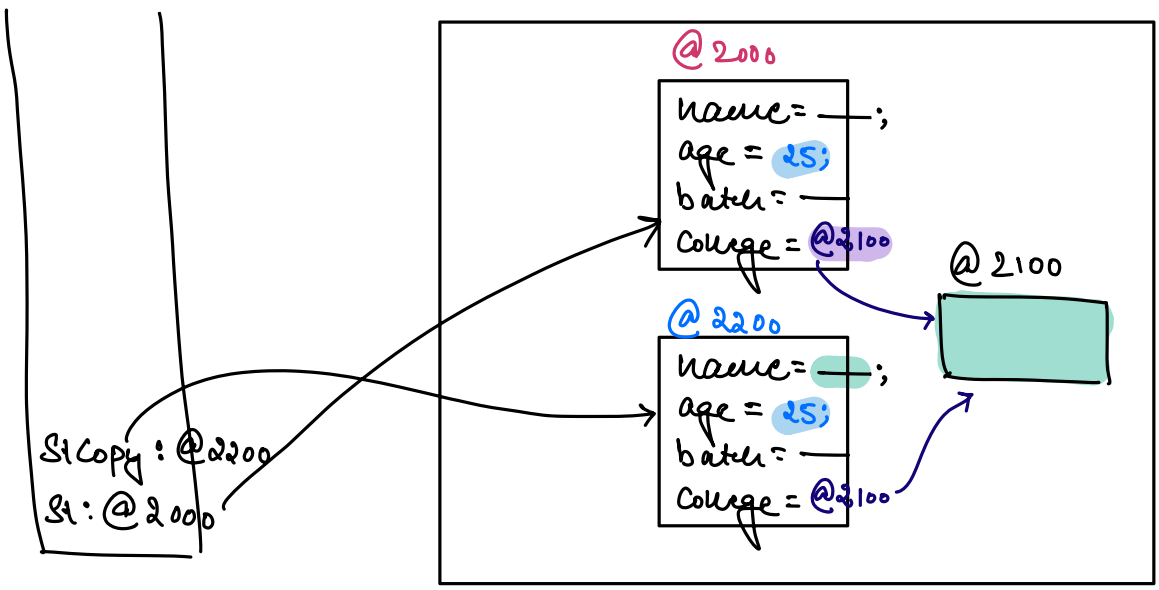
```
Student (Student s) {
    this.name = s.name;
    this.age = s.age;
    this.psp = psp;
}
```

COPY Constructor

```
College {
    ==
}
```

```
College college = new College();
```

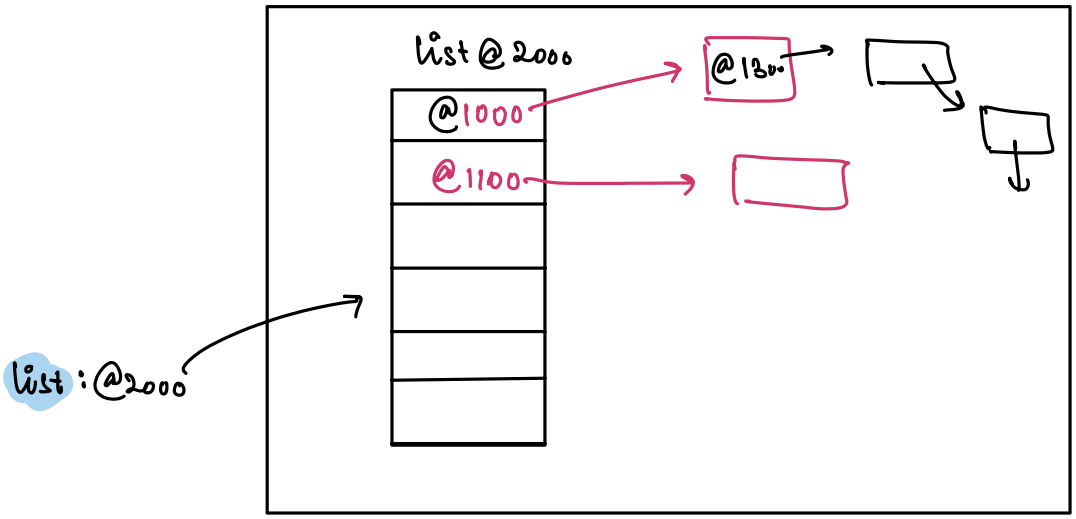
```
St.setCollege(college);
```



⇒ Shallow Copy.

Student StCopy = new Student (st);

⇒ Deep copy.



Pass By Value (vs) Pass By Reference.

```
fun(int x) {
    x += 100;
}
```

x = 10 → 110

```
int x = 10;
fun(x);
```

x (10) @78

```
sout(x)
```

↳ 10

⇒ Pass By Value

C++

Pass By Reference.

```
fun(int &x) {
    x += 100;
}
int x = 10;
fun(x);
cout(x)
```

@78

x @ 78 → 10 → 110

↳ 110.

JAVA.

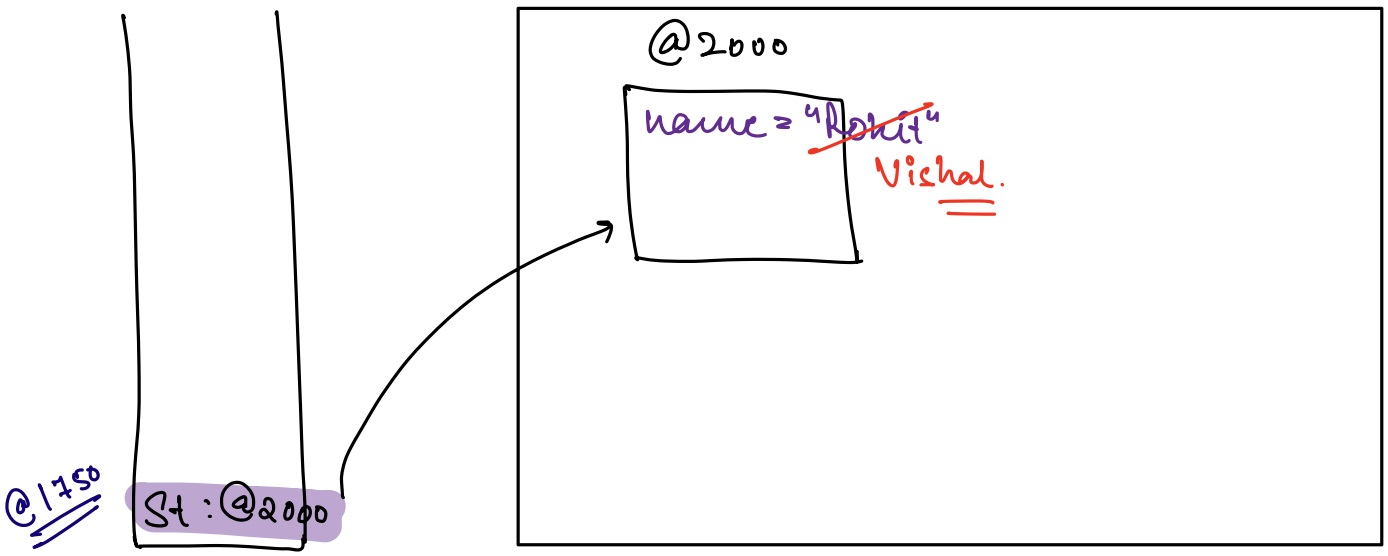
↳ Pass By Value.

```
fun(Student st) {
    st.name = "Vishal";
}
Student st = new Student();
st.name = "Rohit"
fun(st);
Sout(st.name);
```

@2000

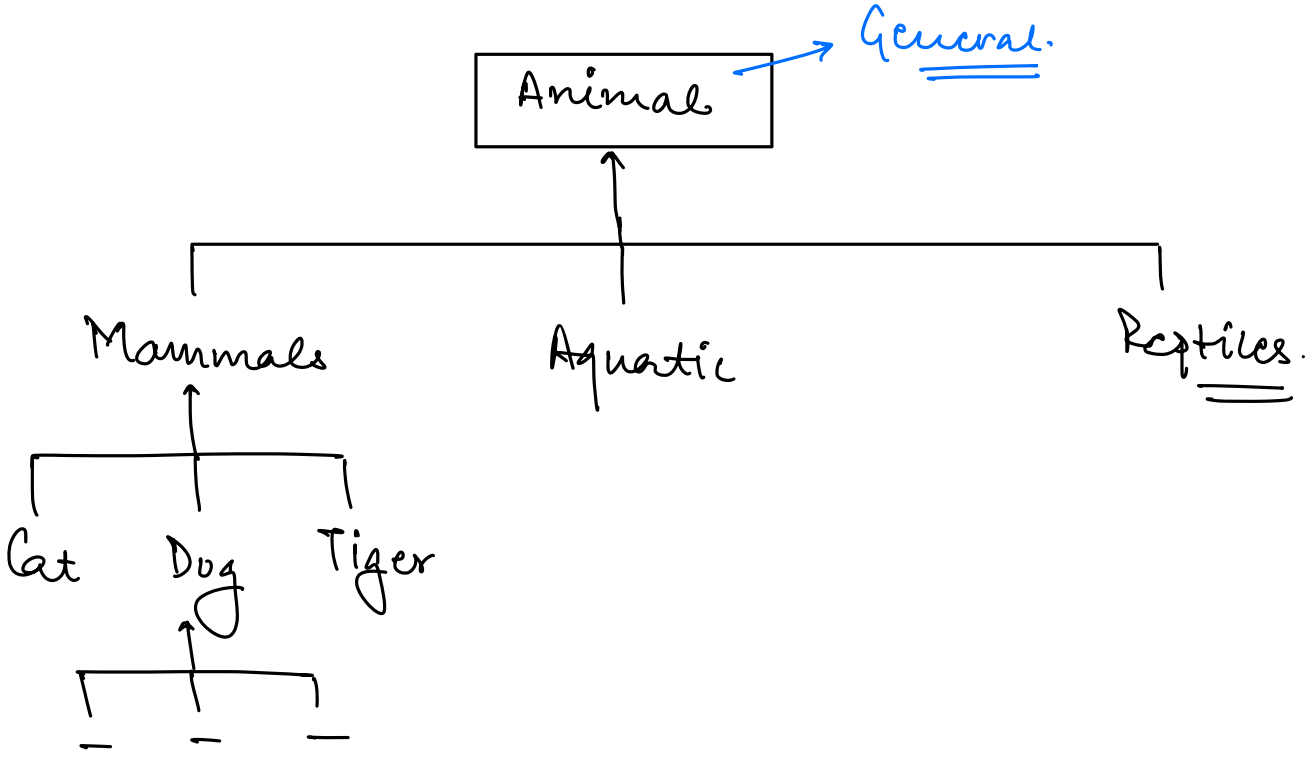
@2000

↳ Vishal.



Inheritance.

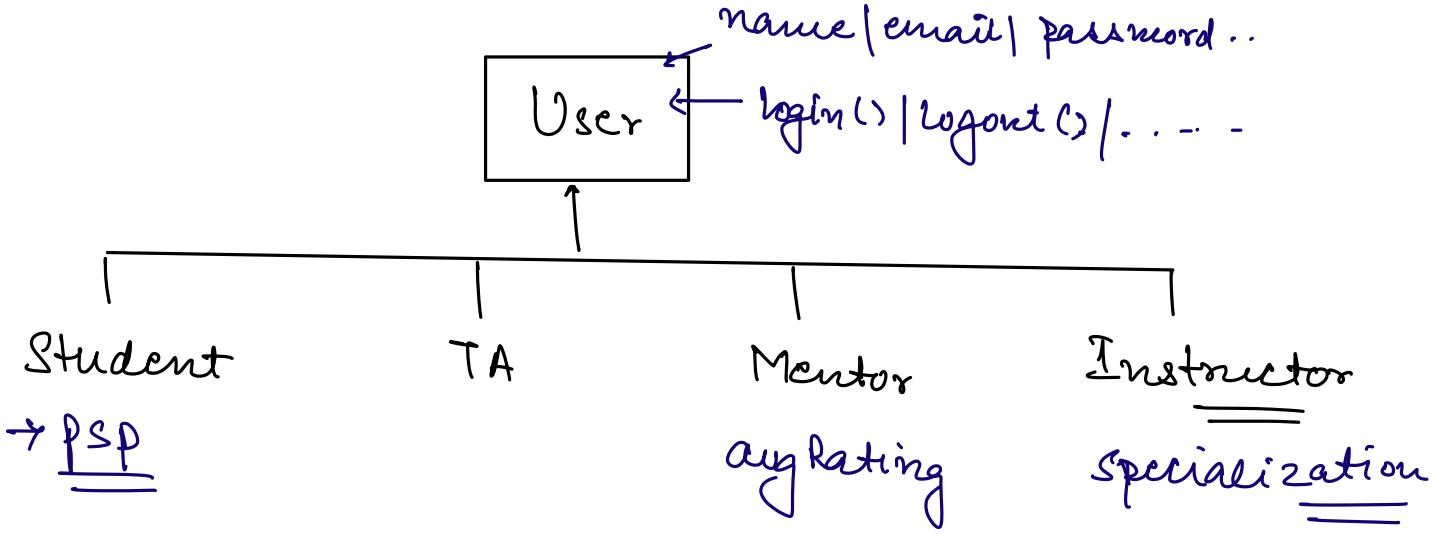
↳ Hierarchy.

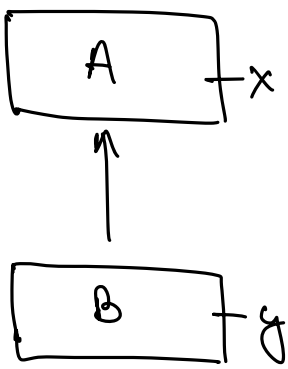


⇒ Parent Child relation.

⇒ Code reusability.

⇒ Every child class will inherit the attrs & behaviours from the parent, also child can have its own attrs & behaviours.

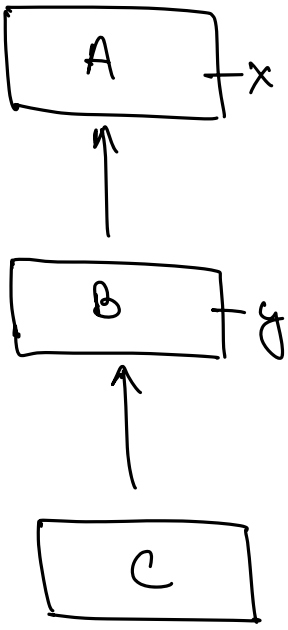




A a = new A(); ✓

B b = new B();

first it invokes the default constructor of its parent.



C c = new C()

C()
↓
B()
↓
A()

⇒ Constructor Chaining.

Super
↓
Refers to parent class constructor.

→ It should always be the first line in the constructor.